



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Programación Comercial 1 [Sec.]

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD 1: Base de datos

Introducción a las bases de datos

Agenda

-  Introducción a SQL.
-  Creación de bases de datos y tablas.
-  Uso de llaves primarias y foráneas.
-  Inserción, modificación y eliminación de datos.
-  Consultas básicas con SELECT.
-  Relación de SQL con el proyecto final.

COMPETENCIA(S) QUE DESARROLLAREMOS



1. Comprende el uso básico de SQL en bases de datos.
2. Crea bases de datos, tablas y relaciones.
3. Aplica llaves primarias, foráneas y restricciones.
4. Inserta, modifica, elimina y consulta datos.
5. Relaciona los comandos SQL con el desarrollo del proyecto final.

¿Qué es SQL?

SQL es el lenguaje que usamos para hablar con la base de datos. Con SQL podemos crear bases y tablas, guardar información, buscar datos, modificarlos y eliminarlos. En otras palabras, SQL es la forma escrita de lo que antes hacían con clics en modo gráfico.



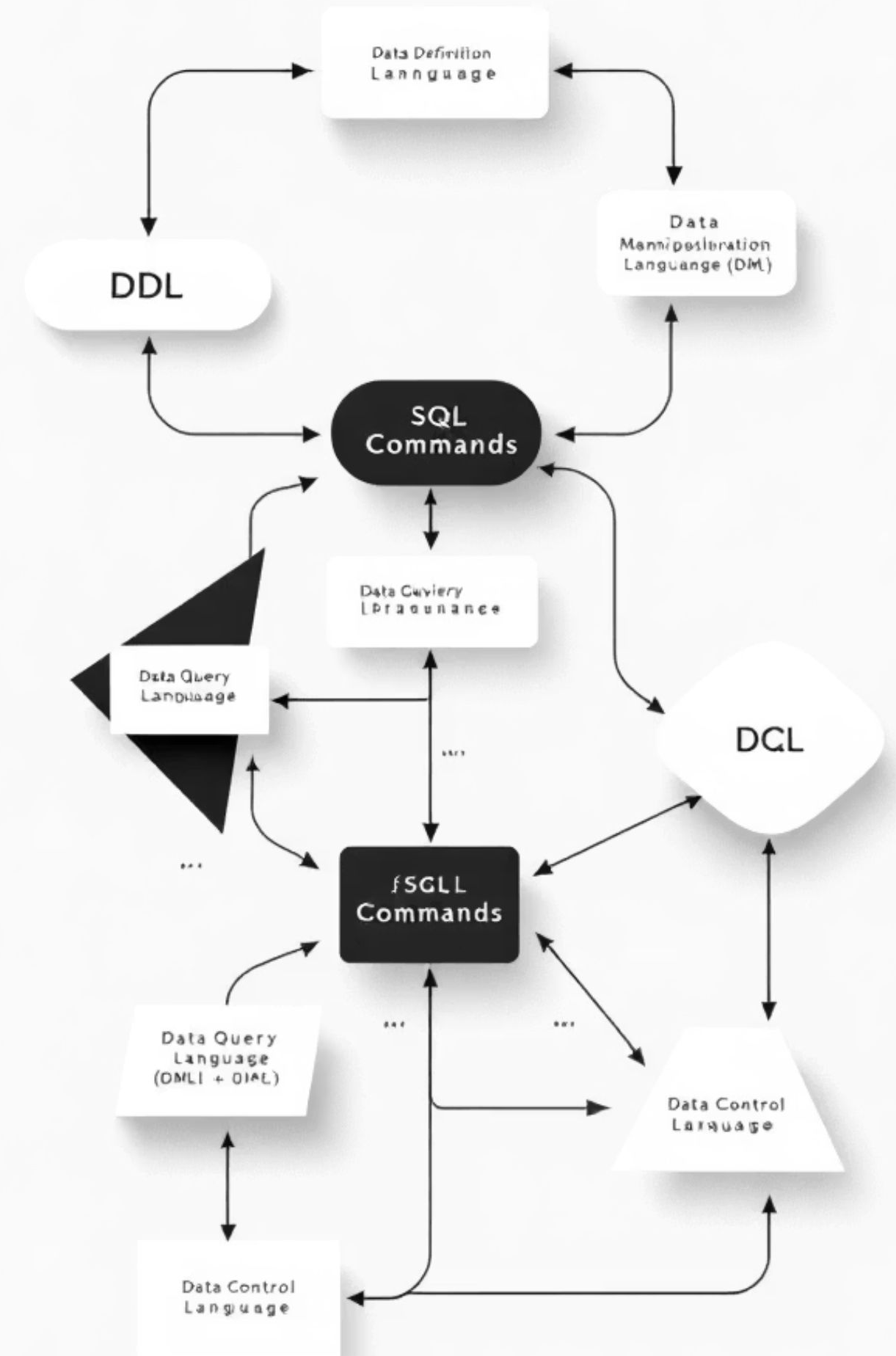
Idea clave de la clase

Lo que antes hacían dibujando tablas y relaciones, ahora lo escribirán con instrucciones.

Si antes relacionaban Cliente con Factura de forma visual, ahora esa misma idea se escribe con comandos como CREATE TABLE, PRIMARY KEY y FOREIGN KEY. El objetivo es que entiendan que no están aprendiendo algo totalmente nuevo, sino otra forma de representar lo mismo.

Cómo pensar SQL sin complicarse

SQL se puede entender por familias de comandos. Unos sirven para crear estructura, otros para trabajar con datos y otros para consultar información. Si aprenden esa lógica, ya no verán SQL como una lista desordenada de palabras raras.



Comandos para crear estructura

Aquí entran los comandos que construyen la base de datos y sus tablas: CREATE DATABASE, USE, CREATE TABLE, PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK. Estos comandos son los más importantes para empezar el proyecto, porque con ellos construyen la base correctamente desde el inicio.

CREATE DATABASE

Crea la base de datos principal

CREATE TABLE

Define la estructura de las tablas

PRIMARY KEY

Establece identificadores únicos

FOREIGN KEY

Crea relaciones entre tablas

Comandos para trabajar con datos

Una vez creada la estructura, necesitamos meter y modificar información. Para eso existen INSERT, UPDATE y DELETE. Estos no crean tablas ni relaciones, sino que trabajan con los registros dentro de las tablas.

INSERT

Agrega nuevos registros a las tablas

UPDATE

Modifica datos existentes en los registros

DELETE

Elimina registros específicos de las tablas

Comandos para consultar información

Para hacer preguntas a la base usamos principalmente SELECT, FROM, WHERE, ORDER BY, JOIN, GROUP BY y HAVING. Estos comandos serán muy importantes porque con ellos podrán resolver las consultas que les pedirá el proyecto.



SELECT & FROM

Especifica qué datos queremos y de dónde



WHERE

Aplica condiciones para filtrar resultados



JOIN

Conecta información de múltiples tablas



ORDER BY

Ordena los resultados de forma ascendente o descendente



GROUP BY & HAVING

Agrupar datos y aplicar condiciones a grupos

CREATE DATABASE

Este comando crea una base de datos nueva. Se usa al inicio, porque primero necesito el contenedor general donde vivirán las tablas. Es como crear la carpeta principal del proyecto.

Ejemplo:

```
CREATE DATABASE MiTiendaDB;
```



Comandos SQL para Bases de Datos

USE

Este comando indica en qué base de datos voy a trabajar. Se usa después de crear la base o cuando quiero cambiar a otra. Es importante porque evita que cree tablas en el lugar equivocado.

Ejemplo:

```
USE MiTiendaDB;
```



GO

GO no crea ni modifica datos; sirve para separar bloques de ejecución. Ayuda a organizar el script y a ejecutar una parte antes de pasar a la siguiente.

Ejemplo:

```
CREATE DATABASE MiTiendaDB;
```

```
GO
```

```
USE MiTiendaDB;
```

```
GO
```

CREATE TABLE

Este comando crea una tabla. Se usa cuando ya sé qué entidad necesito guardar, por ejemplo productos, clientes o facturas. Cada tabla representa un tipo específico de información del negocio.

Ejemplo:

```
CREATE TABLE Producto (  
  IdProducto INT,  
  NombreProducto NVARCHAR(100),  
  PrecioUnitario DECIMAL(10,2)  
);
```

IDENTITY

IDENTITY sirve para que el ID se genere automáticamente. Se usa cuando quiero que cada registro tenga un número único sin escribirlo manualmente. Es útil porque evita errores y repeticiones en los identificadores.

Ejemplo:

```
IdProducto INT IDENTITY(1,1) NOT NULL
```

PRIMARY KEY

La llave primaria identifica de forma única cada fila de una tabla. Se usa en toda tabla principal porque evita que existan dos registros con la misma identidad. Es como el DPI de cada fila.

Ejemplo:

```
CONSTRAINT PK_Producto PRIMARY KEY (IdProducto)
```

FOREIGN KEY

La llave foránea conecta tablas entre sí. Se usa cuando una tabla depende de otra, por ejemplo una factura depende de un cliente. Sirve para mantener la integridad referencial, o sea que la relación sea válida y no inventada.

Ejemplo:

```
CONSTRAINT FK_Factura_Cliente  
FOREIGN KEY (IdCliente)  
REFERENCES Cliente(IdCliente)
```


NOT NULL

Esta restricción indica que un campo es obligatorio. Se usa cuando un dato no puede faltar, por ejemplo el nombre de un producto.

Ayuda a evitar registros incompletos.

Ejemplo:

```
NombreProducto NVARCHAR(100) NOT NULL
```

UNIQUE

Esta restricción impide que se repitan valores en una columna. Se usa cuando un dato debe ser único, por ejemplo un correo o algún código específico. No siempre se necesita, pero cuando aplica ayuda a mantener consistencia.

Ejemplo:

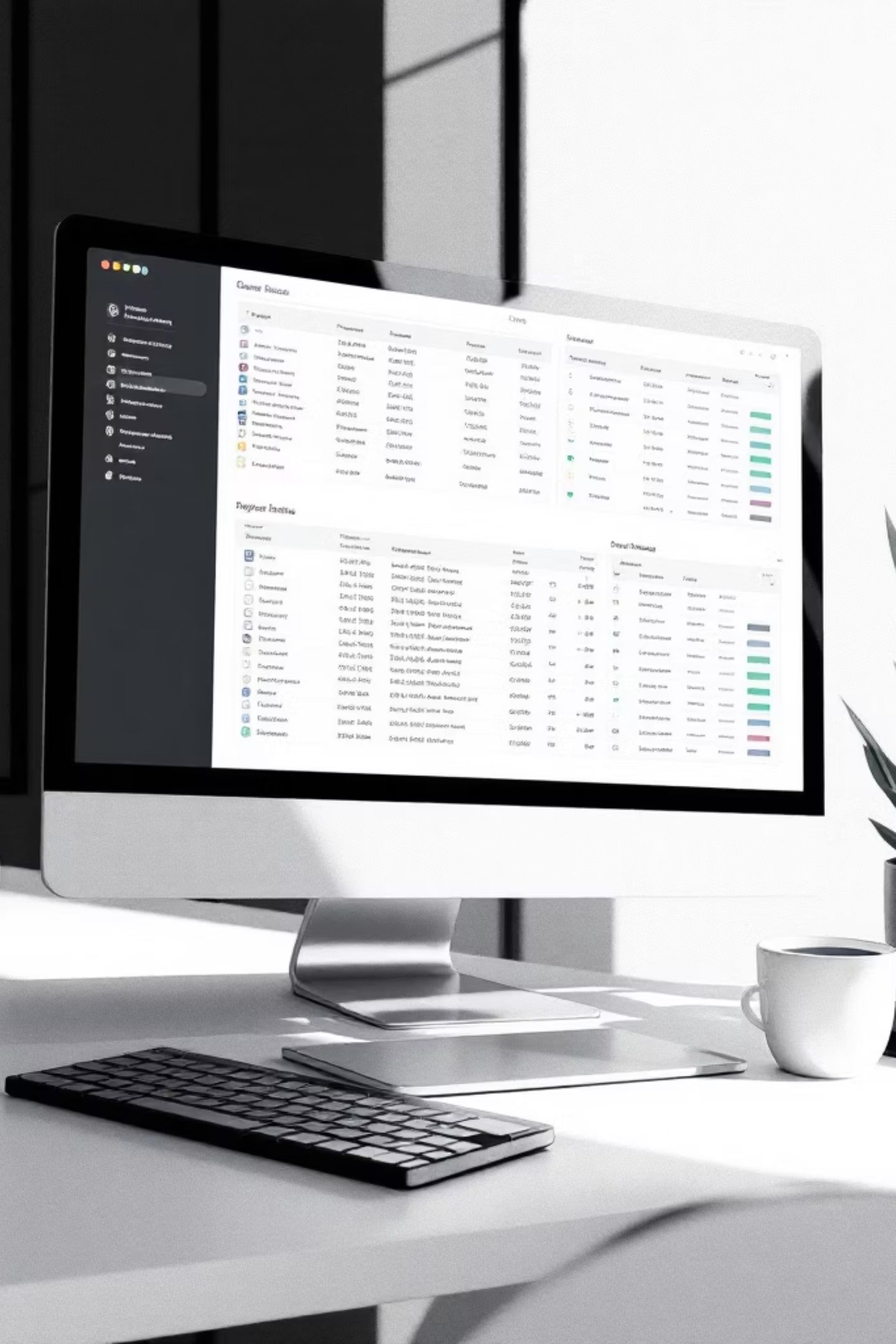
```
Telefono NVARCHAR(20) UNIQUE
```

CHECK

Esta restricción pone una regla para validar valores. Se usa cuando quiero evitar datos incorrectos, por ejemplo precios negativos o cantidades menores a cero. Sirve para proteger la calidad de la información desde la estructura.

Ejemplo:

```
CONSTRAINT CK_Producto_Precio CHECK (PrecioUnitario >= 0)
```



Comandos SQL Básicos

Una guía práctica para manipular y consultar datos en bases de datos relacionales.

INSERT

INSERT agrega registros nuevos a una tabla. Se usa cuando la estructura ya existe y ahora quiero guardar información. Una tabla vacía no sirve para consultar, por eso este comando es básico.

Ejemplo:

```
INSERT INTO Producto (NombreProducto, PrecioUnitario, Stock)
VALUES ('Cemento', 65.00, 20);
```


SELECT

SELECT sirve para consultar o mostrar información. Es el comando más usado cuando queremos ver datos. Casi todas las preguntas que se le hacen a una base comienzan con SELECT.

Ejemplo:

```
SELECT * FROM Producto;
```


FROM

FROM indica de qué tabla saldrán los datos. Siempre acompaña a SELECT, porque no basta con decir qué quiero ver; también tengo que indicar de dónde lo voy a sacar.

Ejemplo:

```
SELECT NombreProducto  
FROM Producto;
```

WHERE

WHERE sirve para filtrar filas según una condición. Se usa cuando no quiero ver todos los registros, sino solo los que cumplen cierta regla. Es uno de los comandos más útiles para búsquedas específicas.

Ejemplo:

```
SELECT *  
FROM Producto  
WHERE Stock < 10;
```

ORDER BY

Este comando ordena los resultados. Se usa cuando quiero ver información de mayor a menor, de menor a mayor o alfabéticamente. Es importante para interpretar mejor los datos y para hacer rankings.

Ejemplo:

```
SELECT NombreProducto, PrecioUnitario  
  
FROM Producto  
  
ORDER BY PrecioUnitario DESC;
```

JOIN

JOIN une tablas relacionadas. Se usa porque en una base relacional la información está separada en varias tablas para no repetir datos. Luego, cuando quiero consultar información completa, necesito volver a unir esas tablas.

Ejemplo:

```
SELECT f.IdFactura, c.NombreCliente  
FROM Factura f  
INNER JOIN Cliente c  
ON f.IdCliente = c.IdCliente;
```

GROUP BY

GROUP BY agrupa filas para resumir información. Se usa cuando quiero contar, sumar o sacar promedios por grupos, por ejemplo ventas por sucursal o totales por cliente. Es clave para las consultas de análisis.

Ejemplo:

```
SELECT IdSucursal, SUM(Total) AS Ventas  
FROM Factura  
GROUP BY IdSucursal;
```


HAVING

HAVING filtra grupos ya agrupados. Se usa después de GROUP BY cuando quiero quedarme solo con ciertos resultados resumidos. La diferencia es que WHERE filtra filas y HAVING filtra grupos.

Ejemplo:

```
SELECT IdSucursal, SUM(Total) AS Ventas  
FROM Factura  
GROUP BY IdSucursal  
HAVING SUM(Total) > 1000;
```

UPDATE

UPDATE modifica registros existentes. Se usa cuando un dato cambia o estaba incorrecto. No cambia la estructura de la tabla, solo el contenido de las filas.

Ejemplo:

```
UPDATE Producto  
SET PrecioUnitario = 70.00  
WHERE IdProducto = 1;
```




DELETE y Archivos .sql

DELETE elimina registros. Se usa cuando un dato ya no debe estar en la tabla. Aquí hay una advertencia importante: sin WHERE, puede borrar todos los registros de la tabla.

Ejemplo:

```
DELETE FROM Producto  
WHERE IdProducto = 1;
```



Cómo leer un archivo .sql sin asustarse

Un archivo .sql no se debe leer todo de golpe. Lo correcto es leerlo por bloques. Primero reviso si crea la base, luego si usa esa base, después si crea las tablas, luego si define PK y FK, y finalmente si agrega validaciones o inserta datos. Si lo leo por partes, se vuelve mucho más fácil de entender.

Qué revisar para validar un .sql

Al revisar un script, pregúntense:

¿Crea la base correcta?

¿Usa la base correcta?

¿Las tablas están en orden lógico?

¿Cada tabla tiene PK?

¿Las FK apuntan bien?

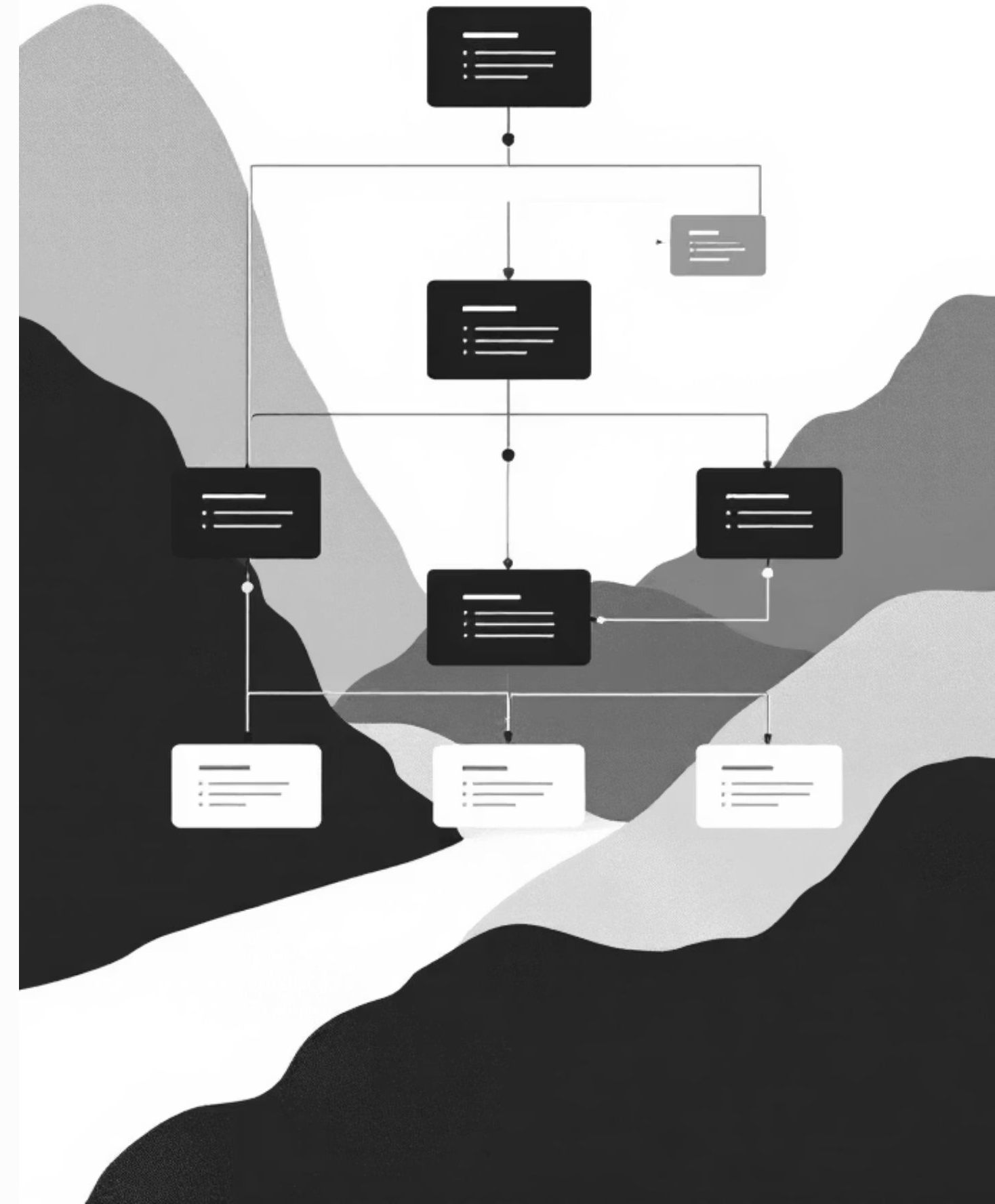
¿Hay restricciones como NOT NULL, CHECK o UNIQUE?

¿Corre sin errores?

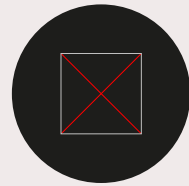
Ese pequeño checklist ya sirve bastante para detectar problemas antes de entregar.

Orden lógico del script

No se escriben las tablas en cualquier orden. Primero van las tablas padre, luego las tablas hijas. Por ejemplo, primero se crea Cliente y después Factura, porque Factura depende de Cliente. Ese orden evita errores con las llaves foráneas.

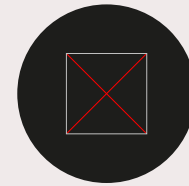


Buenas prácticas al escribir scripts



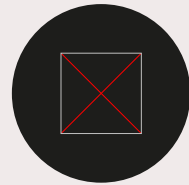
Nombres claros

Usen nombres claros



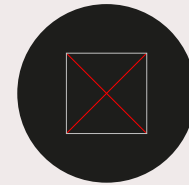
Orden lógico

Mantengan un orden lógico



Separar bloques

separen bloques con GO cuando haga sentido



Comentarios

Agreguen comentarios simples si es necesario

Un script ordenado es más fácil de entender, corregir y revisar. No se trata solo de que funcione, sino también de que sea claro.

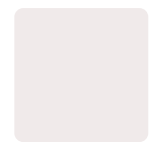
Cómo se conecta todo esto con su proyecto

En su proyecto necesitarán crear la base de datos, crear tablas, definir llaves primarias y foráneas, poner restricciones, cargar datos y resolver consultas. Todo lo que vimos hoy no es teoría suelta: son justamente las herramientas mínimas que necesitan para empezar y avanzar bien.

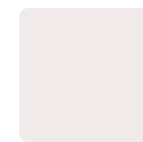


Ejemplos de preguntas que pueden resolver con SQL

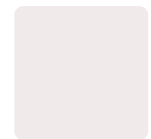
Con estos comandos ya pueden empezar a responder preguntas como:



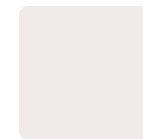
Cuáles productos tienen poco stock



Cuánto se vendió por sucursal

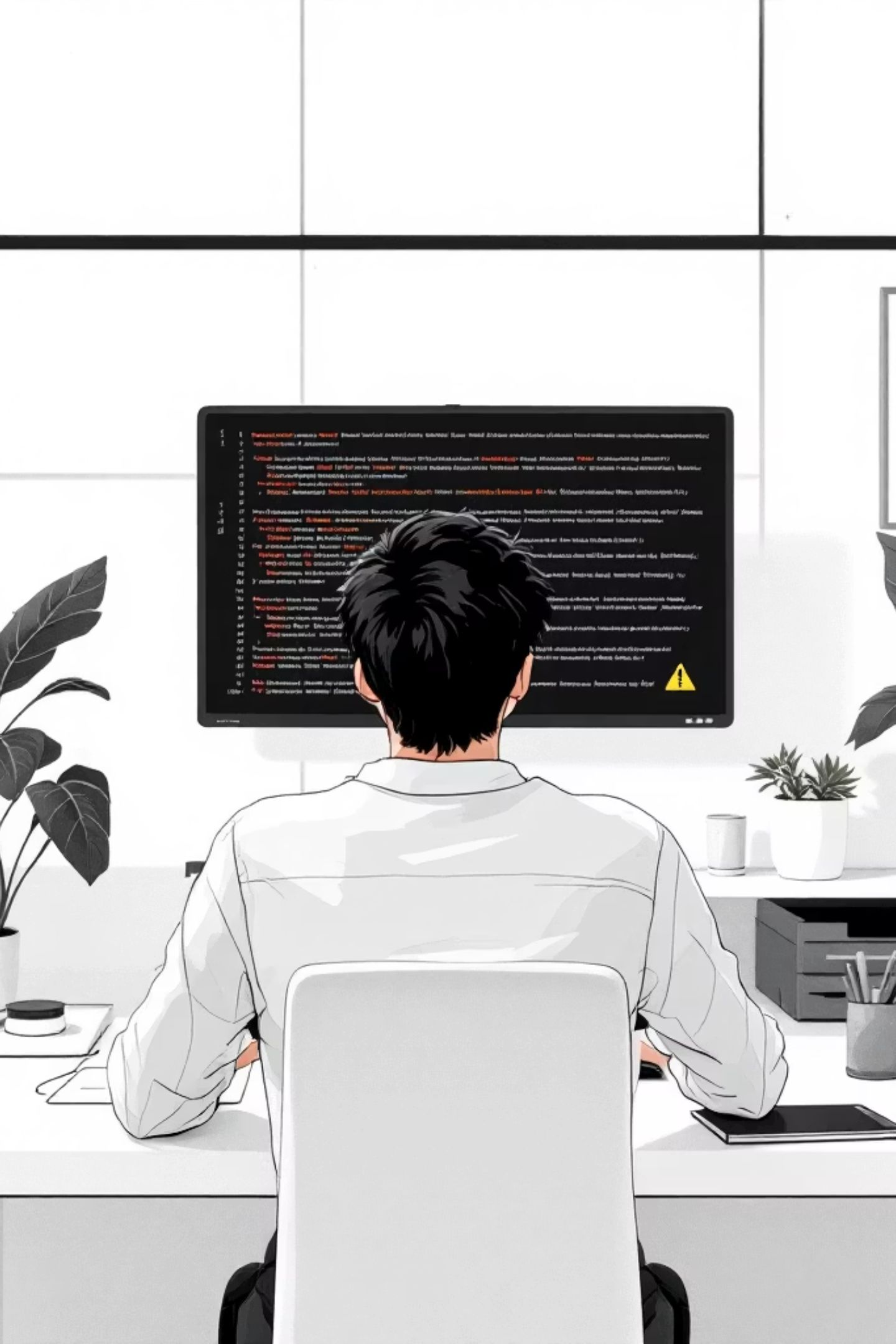


Qué clientes compraron más



Qué productos se venden más

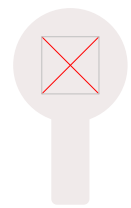
La clave está en combinar correctamente SELECT, WHERE, JOIN, GROUP BY y ORDER BY.



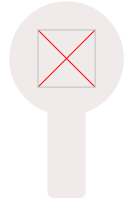
Qué no deben hacer todavía

No empiecen complicándose con temas avanzados si aún no dominan lo básico. Primero asegúrense de que su base, tablas, relaciones y validaciones funcionen bien. Después pasen a consultas más complejas. Tampoco copien código sin entenderlo, porque luego no sabrán corregir errores.

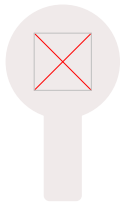
Ruta recomendada para iniciar el proyecto



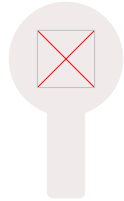
Primero creen la base de datos



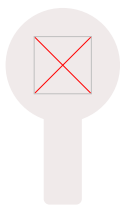
Luego creen las tablas



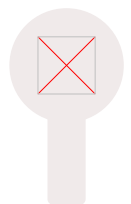
Después agreguen PK y FK



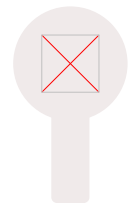
Luego agreguen las restricciones



Después inserten o importen datos



Luego prueben consultas simples



Cuando eso funcione, ya pasan a las consultas del proyecto y a la documentación final



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA